JypyterLabForProject

July 18, 2026

1 Test Environment for Generative AI classroom labs

This lab provides a test environment for the codes generated using the Generative AI classroom.

Follow the instructions below to set up this environment for further use.

Please note, at any point in time, to use the Generative AI tool, click on `TAI` the teaching assistant in the laft bar.

2 Setup

2.0.1 Install required libraries

In case of a requirement of installing certain python libraries for use in your task, you may do so as shown below.

[2]:
```
%pip install pandas
%pip install seaborn
```

```
Collecting pandas
  Downloading pandas-3.0.3-cp312-cp312-
manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl.metadata (79 kB)
Collecting numpy>=1.26.0 (from pandas)
  Downloading
numpy-2.5.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata
(6.6 kB)
Requirement already satisfied: python-dateutil>=2.8.2 in
/opt/conda/lib/python3.12/site-packages (from pandas) (2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.8.2->pandas) (1.17.0)
Downloading
pandas-3.0.3-cp312-cp312-manylinux_2_24_x86_64.manylinux_2_28_x86_64.whl (10.9
MB)
                        10.9/10.9 MB
121.3 MB/s eta 0:00:00
Downloading
numpy-2.5.1-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (16.7
MB)
                        16.7/16.7 MB
32.1 MB/s eta 0:00:00:00:01
```

```
Installing collected packages: numpy, pandas
Successfully installed numpy-2.5.1 pandas-3.0.3
Note: you may need to restart the kernel to use updated packages.
Collecting seaborn
  Downloading seaborn-0.13.2-py3-none-any.whl.metadata (5.4 kB)
Requirement already satisfied: numpy!=1.24.0,>=1.20 in
/opt/conda/lib/python3.12/site-packages (from seaborn) (2.5.1)
Requirement already satisfied: pandas>=1.2 in /opt/conda/lib/python3.12/site-
packages (from seaborn) (3.0.3)
Collecting matplotlib!=3.6.1,>=3.4 (from seaborn)
  Downloading matplotlib-3.11.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (80 kB)
Collecting contourpy>=1.0.1 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading contourpy-1.3.3-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (5.5 kB)
Collecting cycler>=0.10 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading cycler-0.12.1-py3-none-any.whl.metadata (3.8 kB)
Collecting fonttools>=4.22.0 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading fonttools-4.63.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (118 kB)
Collecting kiwisolver>=1.3.1 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading kiwisolver-1.5.0-cp312-cp312-
manylinux2014_x86_64.manylinux_2_17_x86_64.whl.metadata (5.1 kB)
Requirement already satisfied: packaging>=20.0 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(24.2)
Collecting pillow>=9 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading pillow-12.3.0-cp312-cp312-
manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl.metadata (9.1 kB)
Collecting pyparsing>=3 (from matplotlib!=3.6.1,>=3.4->seaborn)
  Downloading pyparsing-3.3.2-py3-none-any.whl.metadata (5.8 kB)
Requirement already satisfied: python-dateutil>=2.7 in
/opt/conda/lib/python3.12/site-packages (from matplotlib!=3.6.1,>=3.4->seaborn)
(2.9.0.post0)
Requirement already satisfied: six>=1.5 in /opt/conda/lib/python3.12/site-
packages (from python-dateutil>=2.7->matplotlib!=3.6.1,>=3.4->seaborn) (1.17.0)
Downloading seaborn-0.13.2-py3-none-any.whl (294 kB)
Downloading
matplotlib-3.11.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl
(10.0 MB)
                         10.0/10.0 MB
49.4 MB/s eta 0:00:0000:01
Downloading
contourpy-1.3.3-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (362
kB)
Downloading cycler-0.12.1-py3-none-any.whl (8.3 kB)
Downloading
fonttools-4.63.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (5.0
```

```
MB)
                           5.0/5.0 MB
64.4 MB/s eta 0:00:00
Downloading
kiwisolver-1.5.0-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.whl (1.5
MB)
                           1.5/1.5 MB
55.1 MB/s eta 0:00:00
Downloading
pillow-12.3.0-cp312-cp312-manylinux_2_27_x86_64.manylinux_2_28_x86_64.whl (6.9
MB)
                           6.9/6.9 MB
145.3 MB/s eta 0:00:00
Downloading pyparsing-3.3.2-py3-none-any.whl (122 kB)
Installing collected packages: pyparsing, pillow, kiwisolver, fonttools, cycler,
contourpy, matplotlib, seaborn
Successfully installed contourpy-1.3.3 cycler-0.12.1 fonttools-4.63.0
kiwisolver-1.5.0 matplotlib-3.11.0 pillow-12.3.0 pyparsing-3.3.2 seaborn-0.13.2
Note: you may need to restart the kernel to use updated packages.
```

2.0.2 Dataset URL from the GenAI lab

Use the URL provided in the GenAI lab in the cell below.

[3]:
```python
import pandas as pd

# Specify the file path to the CSV file
file_path = "2016.csv"  # Replace with your actual file path

# Read the CSV file into a pandas DataFrame
df = pd.read_csv(file_path)

# Print the first 5 rows of the DataFrame
print(df.head())
```

```
       Country          Region  Happiness Rank  Happiness Score  \
0      Denmark  Western Europe               1            7.526
1  Switzerland  Western Europe               2            7.509
2      Iceland  Western Europe               3            7.501
3       Norway  Western Europe               4            7.498
4      Finland  Western Europe               5            7.413

   Lower Confidence Interval  Upper Confidence Interval  \
0                      7.460                      7.592
1                      7.428                       7.59
2                      7.333                      7.669
3                      7.421                      7.575
4                      7.351                      7.475
```

```
  Economy (GDP per Capita)   Family Health (Life Expectancy)  Freedom  \
0                  1.44178  1.16374                  0.79504  0.57941
1                  1.52733  1.14524                  0.86303  0.58557
2                  1.42666  1.18326                  0.86733  0.56624
3                  1.57744  1.12690                  0.79579  0.59609
4                  1.40598  1.13464                  0.81091  0.57104

   Trust (Government Corruption)  Generosity  Dystopia Residual
0                        0.44453     0.36171            2.73939
1                        0.41203     0.28083            2.69463
2                        0.14975     0.47678            2.83137
3                        0.35776     0.37895            2.66465
4                        0.41004     0.25492            2.82596
```

[4]:
```python
import pandas as pd
import numpy as np



# ------------------------------
# 2. Replace empty strings with NaN
# ------------------------------
df.replace(r'^\s*$', np.nan, regex=True, inplace=True)

# ------------------------------
# 3. Remove leading and trailing whitespaces
#    from all string (object/string) columns
# ------------------------------
string_cols = df.select_dtypes(include=["object", "string"]).columns

for col in string_cols:
    df[col] = df[col].str.strip()

# Replace empty strings again after stripping whitespace
df.replace(r'^\s*$', np.nan, regex=True, inplace=True)

# ------------------------------
# 4. Convert columns to appropriate data types
#    (latest pandas)
# ------------------------------
# Convert object columns to the best possible dtypes
df = df.convert_dtypes()

# Try converting string columns that contain numbers
for col in df.columns:
    if df[col].dtype == "string":
```

```
        converted = pd.to_numeric(df[col], errors="coerce")

        # Convert only if most non-null values are numeric
        if converted.notna().sum() >= 0.8 * df[col].notna().sum():
            df[col] = converted

# Convert dtypes again
df = df.convert_dtypes()

# -----------------------------
# 5. Replace missing values
# -----------------------------

# Numeric columns -> replace NaN with column mean
numeric_cols = df.select_dtypes(include="number").columns

for col in numeric_cols:
    df[col] = df[col].fillna(df[col].mean())

# Categorical columns -> replace NaN with column mode
categorical_cols = df.select_dtypes(include=["object", "string", "category"]).
 ↪columns

for col in categorical_cols:
    if not df[col].mode().empty:
        df[col] = df[col].fillna(df[col].mode()[0])

# -----------------------------
# 6. Display information
# -----------------------------
print("First 5 rows of cleaned data:")
print(df.head())

print("\nData Types:")
print(df.dtypes)

print("\nMissing values in each column:")
print(df.isnull().sum())
```

```
First 5 rows of cleaned data:
       Country          Region  Happiness Rank  Happiness Score  \
0      Denmark  Western Europe               1            7.526
1  Switzerland  Western Europe               2            7.509
2      Iceland  Western Europe               3            7.501
3       Norway  Western Europe               4            7.498
4      Finland  Western Europe               5            7.413

   Lower Confidence Interval  Upper Confidence Interval  \
```

```
0                       7.46                       7.592
1                      7.428                        7.59
2                      7.333                       7.669
3                      7.421                       7.575
4                      7.351                       7.475

   Economy (GDP per Capita)   Family  Health (Life Expectancy)  Freedom  \
0                   1.44178  1.16374                   0.79504  0.57941
1                   1.52733  1.14524                   0.86303  0.58557
2                   1.42666  1.18326                   0.86733  0.56624
3                   1.57744   1.1269                   0.79579  0.59609
4                   1.40598  1.13464                   0.81091  0.57104

   Trust (Government Corruption)  Generosity  Dystopia Residual
0                        0.44453     0.36171            2.73939
1                        0.41203     0.28083            2.69463
2                        0.14975     0.47678            2.83137
3                        0.35776     0.37895            2.66465
4                        0.41004     0.25492            2.82596

Data Types:
Country                          string
Region                           string
Happiness Rank                    Int64
Happiness Score                 Float64
Lower Confidence Interval       Float64
Upper Confidence Interval       Float64
Economy (GDP per Capita)        Float64
Family                          Float64
Health (Life Expectancy)        Float64
Freedom                         Float64
Trust (Government Corruption)   Float64
Generosity                      Float64
Dystopia Residual               Float64
dtype: object

Missing values in each column:
Country                          0
Region                           0
Happiness Rank                   0
Happiness Score                  0
Lower Confidence Interval        0
Upper Confidence Interval        0
Economy (GDP per Capita)         0
Family                           0
Health (Life Expectancy)         0
Freedom                          0
Trust (Government Corruption)    0
```

```
Generosity                       0
Dystopia Residual                0
dtype: int64
```

[5]:
```
df.head()
```

[5]:

	Country	Region	Happiness Rank	Happiness Score	\
0	Denmark	Western Europe	1	7.526	
1	Switzerland	Western Europe	2	7.509	
2	Iceland	Western Europe	3	7.501	
3	Norway	Western Europe	4	7.498	
4	Finland	Western Europe	5	7.413	

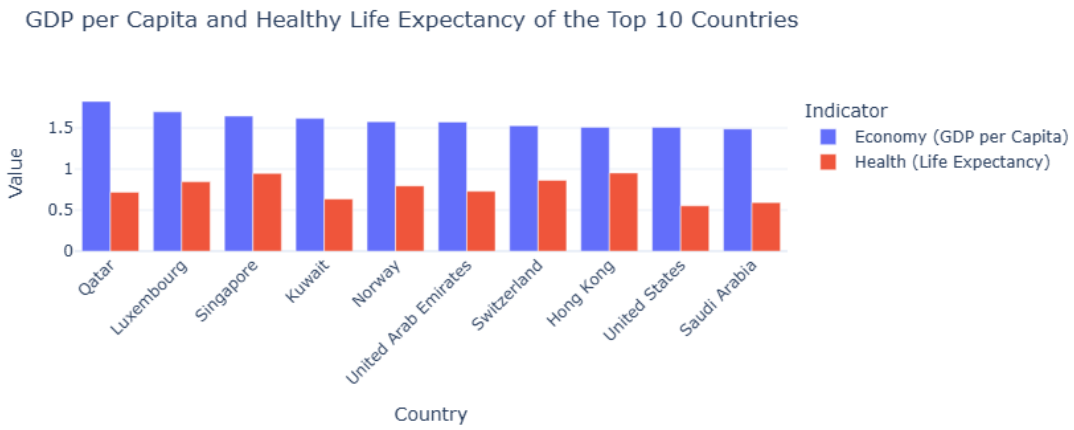
	Lower Confidence Interval	Upper Confidence Interval	\
0	7.46	7.592	
1	7.428	7.59	
2	7.333	7.669	
3	7.421	7.575	
4	7.351	7.475	

	Economy (GDP per Capita)	Family	Health (Life Expectancy)	Freedom	\
0	1.44178	1.16374	0.79504	0.57941	
1	1.52733	1.14524	0.86303	0.58557	
2	1.42666	1.18326	0.86733	0.56624	
3	1.57744	1.1269	0.79579	0.59609	
4	1.40598	1.13464	0.81091	0.57104	

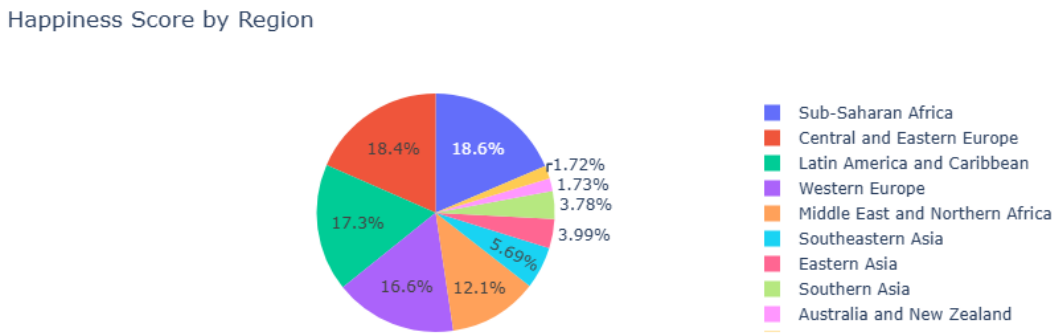
	Trust (Government Corruption)	Generosity	Dystopia Residual
0	0.44453	0.36171	2.73939
1	0.41203	0.28083	2.69463
2	0.14975	0.47678	2.83137
3	0.35776	0.37895	2.66465
4	0.41004	0.25492	2.82596

[11]:
```
import pandas as pd
import plotly.express as px


# Remove leading/trailing spaces from column names
df.columns = df.columns.str.strip()

# Select the top 10 countries by GDP per capita
top10 = (
    df.nlargest(10, "Economy (GDP per Capita)")
    [["Country", "Economy (GDP per Capita)", "Health (Life Expectancy)"]]
)

# Reshape the data for Plotly
```

```
top10_long = top10.melt(
    id_vars="Country",
    value_vars=["Economy (GDP per Capita)", "Health (Life Expectancy)"],
    var_name="Indicator",
    value_name="Value"
)

# Create the grouped bar chart
fig1 = px.bar(
    top10_long,
    x="Country",
    y="Value",
    color="Indicator",
    barmode="group",
    title="GDP per Capita and Healthy Life Expectancy of the Top 10 Countries",
    labels={
        "Country": "Country",
        "Value": "Value",
        "Indicator": "Indicator"
    }
)

# Improve the layout
fig1.update_layout(
    xaxis_tickangle=-45,
    template="plotly_white"
)

# Display the chart
fig1.show()
```

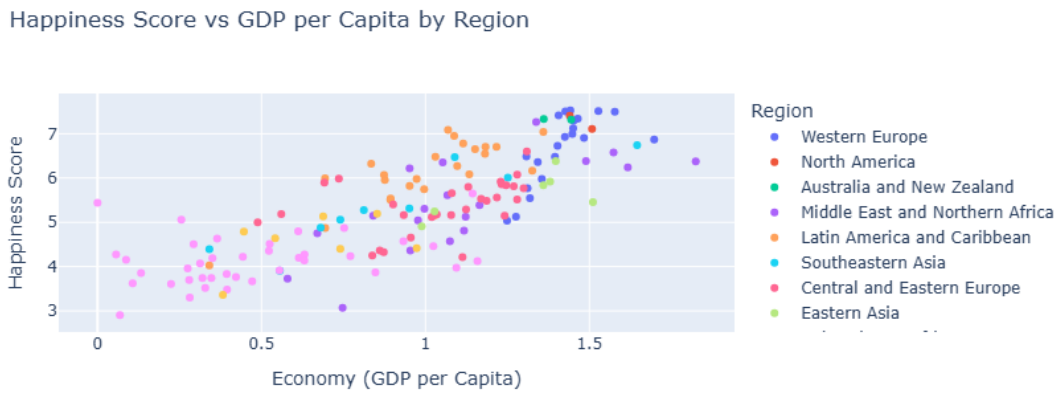


[12]:
```
import pandas as pd
import plotly.express as px


# Remove leading/trailing spaces from column names
df.columns = df.columns.str.strip()

# Group the data by Region and calculate the total Happiness Score
region_happiness = (
    df.groupby("Region", as_index=False)["Happiness Score"]
      .sum()
)

# Create the pie chart
fig4 = px.pie(
    region_happiness,
    names="Region",
    values="Happiness Score",
    title="Happiness Score by Region"
)

# Display the chart
fig4.show()
```



[23]:
```
import pandas as pd
import plotly.express as px

# Read the CSV file

# Remove leading/trailing spaces from column names
```

```
df.columns = df.columns.str.strip()

# Create the scatter plot
fig3 = px.scatter(
    df,
    x="Economy (GDP per Capita)",
    y="Happiness Score",
    color="Region",
    hover_name="Country",
    title="Happiness Score vs GDP per Capita by Region",
    labels={
        "GDP per capita": "GDP per Capita",
        "Happiness Score": "Happiness Score",
        "Region": "Region"
    }
)

# Display the figure
fig3.show()
```



2.0.3 Downloading the dataset

Execute the following code to download the dataset in to the interface.

[ ]:
```
from plotly.subplots import make_subplots
import plotly.graph_objects as go

# Create a dashboard with 2 rows and 2 columns
dashboard = make_subplots(
    rows=2,
    cols=2,
```

```
    subplot_titles=(
        "Figure 1",
        "Figure 3",
        "Figure 4"
    ),
    specs=[
        [{"type": "bar"}, {"type": "scatter"}],
        [{"type": "domain"}, {"type": "choropleth"}]
    ]
)

# Add traces from fig1
for trace in fig1.data:
    dashboard.add_trace(trace, row=1, col=1)

# Add traces from fig3
for trace in fig3.data:
    dashboard.add_trace(trace, row=1, col=2)

# Add traces from fig4
for trace in fig4.data:
    dashboard.add_trace(trace, row=2, col=1)

# Add traces from fig5

# Update dashboard layout
dashboard.update_layout(
    title="World Happiness Dashboard",
    height=900,
    width=1200,
    showlegend=True
)

# Save the dashboard to an HTML file
dashboard.write_html("dashboard.html")

print("Dashboard saved successfully as 'dashboard.html'.")
```

```
Dashboard saved successfully as 'dashboard.html'.
```

3 Test Environment

[ ]:
```
# Keep appending the code generated to this cell, or add more cells below this␣
↪to execute in parts
```

3.1 Authors

Abhishek Gagneja